FROSTLY

Full Project Documentation & User Manual

Storefront architecture · Supabase setup · Security · Deployment · Customer guide · Admin guide

PROJECT	Static HTML, CSS and JavaScript storefront
BACKEND	Supabase Auth, PostgreSQL, RLS, Storage and RPC
HOSTING	Cloudflare Pages with same-origin API proxy
GUIDES	Technical project reference and customer/admin instructions

Prepared from the current Frostly project files

22 October 2026

Contents

- Part I · Project documentation
- Architecture and customer/admin workflows
- Backend, migrations and security
- Local development and Cloudflare deployment
- Configuration and troubleshooting
- Part II · Customer and administrator manual
- Browse, account, profile and checkout
- Orders and ratings
- Admin dashboard and settings

Frostly - Project Documentation

Project type: Static bilingual frozen-food storefront with Supabase-backed commerce features

Frontend: HTML, CSS, browser JavaScript

Hosting: Cloudflare Pages

Database, auth, and file storage: Supabase

Default language: English; Bangla is available from the language selector

1. Project overview

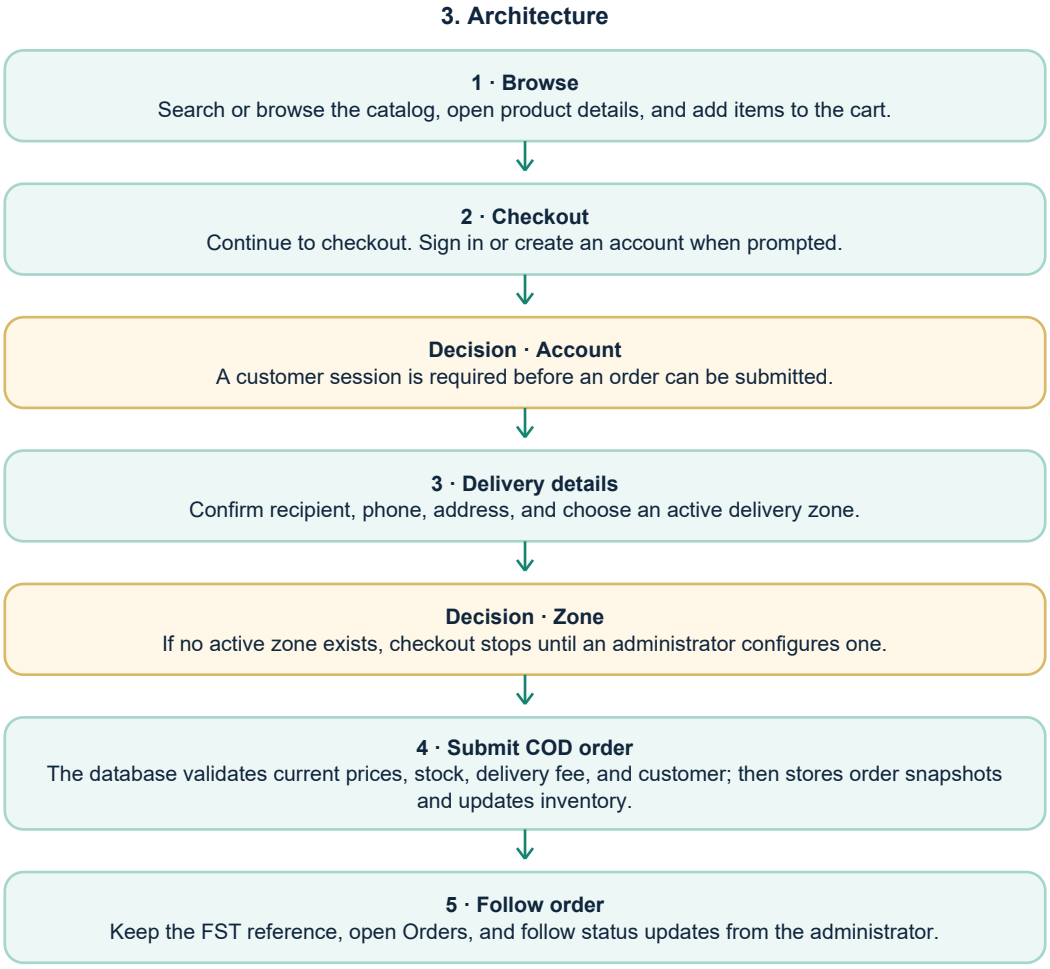
Frostly is a responsive storefront for browsing frozen-food products, using a cart, placing Cash on Delivery (COD) orders, and viewing account/order information. The administrator console manages catalog data, stock, delivery zones, order statuses, and selected storefront settings.

The site is a static frontend, not a server-rendered framework application. Supabase provides the backend services. On Cloudflare Pages, browser Supabase HTTP requests are routed through a same-origin Pages Function reverse proxy. The Supabase JavaScript SDK itself and some imagery/fonts are loaded from external CDNs, so those resources need network access.

2. Repository map

Path	Purpose
`index.html`	Storefront markup, overlays, product detail, account, cart, checkout, orders and assistant panels
`styles.css`	Storefront and responsive styling
`app.js`	Storefront behavior, language, cart, account, checkout, product details, ratings, settings and order UI
`console-a84d8903f59b4d358c8cc927633a11f6.html`	Administrator console document (obscure filename is only a convenience; authorization is enforced separately)
`admin-app.js`	Admin authorization, dashboard data, orders, products, inventory, delivery zones and settings
`supabase-config.js`	Public Supabase project URL and anon/publishable key. Never put a service-role key here.
`supabase-proxy.js`	Rewrites Supabase SDK HTTP requests to the same-origin proxy on deployed HTTPS pages
`functions/api/supabase/[[path]].js`	Cloudflare Pages Function that forwards allowed Supabase API traffic
`_routes.json`	Limits Pages Function invocation to `/api/supabase/*`
`supabase/migrations/`	Ordered SQL schema, policies, RPCs, storage and feature migrations
`README.md`	Short setup overview and migration/admin bootstrap instructions

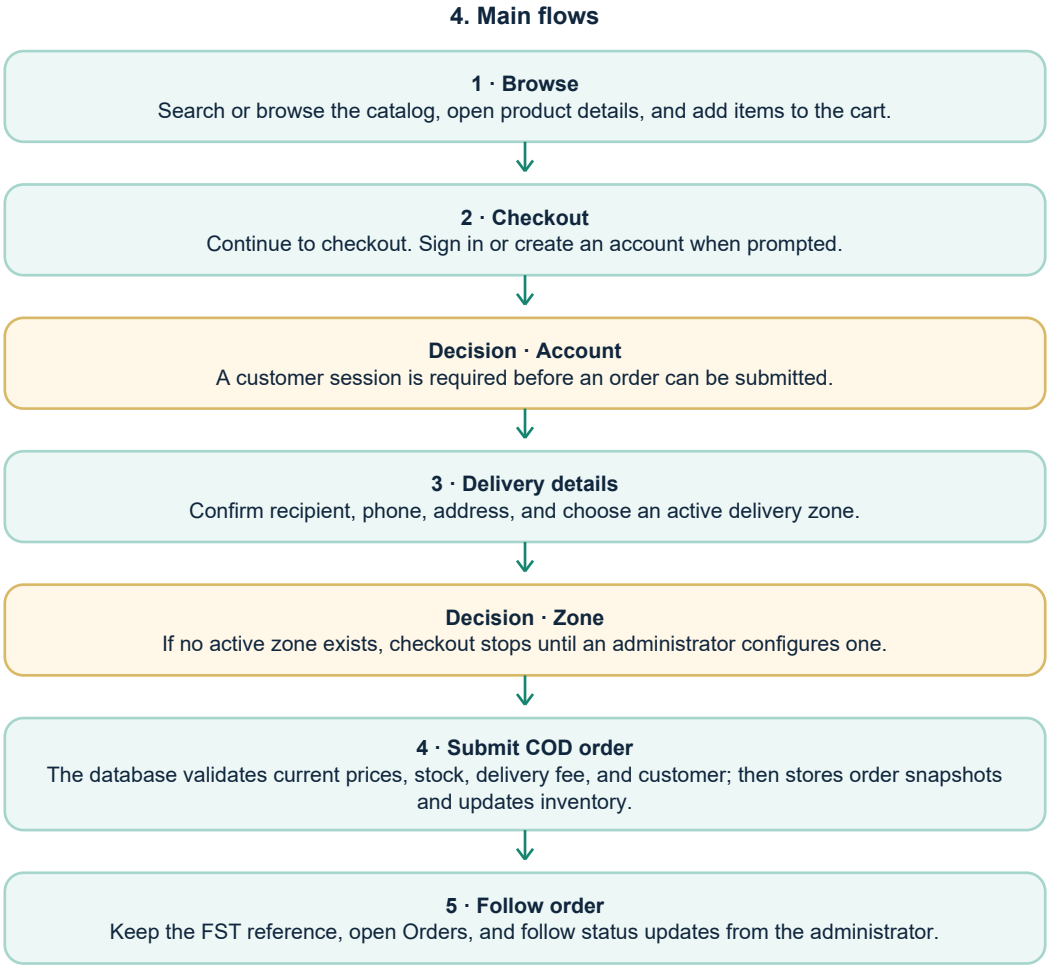
3. Architecture



The browser uses the public anon/publishable key. Row Level Security (RLS), database functions, and role checks must enforce permissions; the key and a hard-to-guess HTML filename do not grant or protect administrator access by themselves. The proxy forwards API calls; it is not an authentication layer and does not replace Supabase policies.

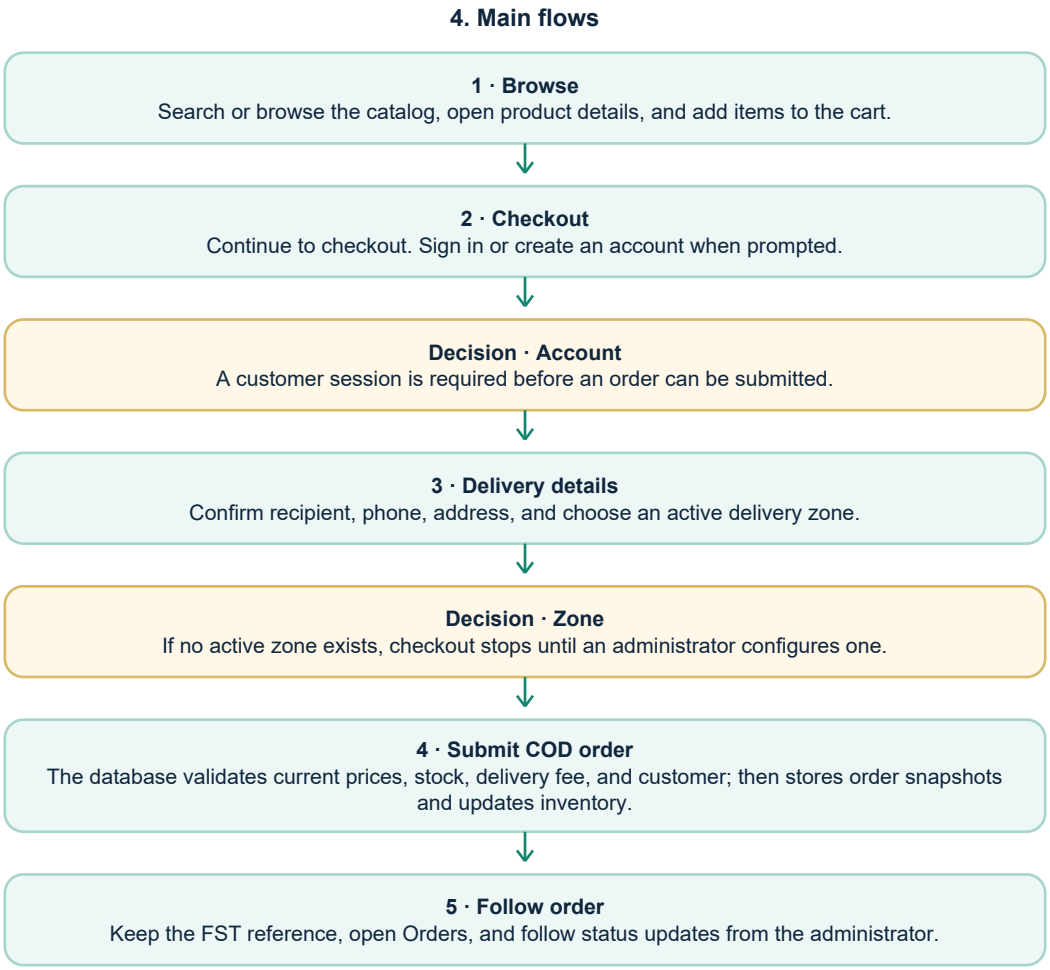
4. Main flows

4.1 Customer order flow



The displayed reference is formatted from the order UUID and date (`FST-YYYY-…`), while the database can retain an internal numeric order number. Do not use the display reference as an authorization credential. Order access is governed by user ownership and database policies.

4.2 Admin workflow



4.3 Review/rating flow

Only an authenticated customer whose order is marked `delivered` and whose order items include the product can create or update one review for that product. Ratings are 1–5 stars and review text is limited to 500 characters. Public users may read reviews. Run migration `202610220009_product_ratings.sql` to enable the feature.

5. Backend and data

Supabase is the system of record for accounts, products, orders, delivery zones, storefront settings, and reviews. Exact columns and constraints are defined in the SQL migrations; consult those files before changing schema or policies.

Migrations (apply in order)

1. `202610200001_frostly_core.sql` — core profiles/catalog/customer data/orders/inventory/status history, RLS and database functions.
2. `202610200002_chattogram_delivery_zones.sql` — active Inside Chattogram Town (Tk 80) and Outside Chattogram Town (Tk 120) delivery zones.
3. `202610200003_admin_inventory_adjustment.sql` — secure admin stock adjustments and movement history.
4. `202610200004_customer_profile_settings.sql` — profile phone/avatar fields and avatar storage permissions.
5. `202610200005_default_delivery_address.sql` — default saved-address support.
6. `202610200006_admin_product_image_uploads.sql` — product image bucket and admin-only upload policies.
7. `202610200007_storefront_settings_slider.sql` — editable brand/logo and homepage slides.
8. `202610200008_site_contact_settings.sql` — editable service area, phone, email and payment display settings.

9. `202610220009_product_ratings.sql` — verified delivered-order product ratings and reviews.

Use the Supabase SQL Editor to run each migration in order for a new project. Existing projects should apply only migrations not already applied. Do not paste Markdown formatting or backticks into the SQL Editor. Review SQL and back up production data before schema changes.

COD checkout integrity

The `create_cod_order` database function is the authoritative checkout operation. It validates the authenticated customer, products and quantities, reads current catalog prices and stock, checks the active delivery zone and fee, stores item snapshots, calculates the total, changes inventory, and uses a request ID for retry idempotency. The client-side total is an estimate only. COD is the configured payment method; no online payment gateway is implemented.

6. Security model and operational cautions

- Keep only the Supabase project URL and public anon/publishable key in browser files. Never publish `service_role`, database passwords, or other secrets.
- Create a normal account first, then grant the first admin role from the trusted Supabase SQL Editor as documented in `README.md`. There is no public admin-signup flow.
- Admin page filename obfuscation is not security. The page checks the logged-in profile role; RLS/database functions must continue to enforce permissions even if someone opens the URL directly.
- Do not disable RLS or grant broad write access to solve a storefront issue. Inspect policy/function errors and use least privilege.
- Public product media and selected storefront settings are intentionally readable. Admin writes/uploads are restricted by policies.
- Verify email confirmation, Site URL, and redirect URLs in Supabase Auth for local and production origins. Password recovery depends on a valid redirect configuration.
- Replace demo products, image URLs, contact details, and any demo copy before operating a real shop. Do not represent demo reviews or stock as verified.
- The customer assistant is catalog-grounded demo behavior, not an OpenAI-backed assistant. Newsletter signup persistence is not wired. No payment gateway, delivery-staff workflow, or full business policy system is implemented.
- Supabase should be treated as a single configured project/database here. No database sharding or read-replica routing is currently implemented.

7. Local development

From the repository root:

```
py -m http.server 8000
```

Open `http://localhost:8000`. Leave the terminal running while using the site; press `Ctrl+C` to stop the server. Local HTTP uses direct Supabase API calls, so verify the project URL/key and Auth redirect settings. If backend configuration/migrations are absent, the static demo portions may render but live authentication, checkout and admin data will not work.

8. Cloudflare Pages deployment

The repository is plain static HTML/JS with a Pages Function. For a Git-connected Pages project:

- Framework preset: **None**
- Build command: leave blank
- Build output directory: `/` (repository root)
- Root directory: repository root, unless the project is intentionally nested
- Ensure `functions/api/supabase/[[path]].js`, `supabase-proxy.js`, and `_routes.json` are committed.
- If the Supabase project URL changes, set Pages variable `FROSTLY_SUPABASE_URL` to the project URL. The proxy otherwise uses its configured project default.

- Push changes to the configured production branch and wait for the Pages deployment to report success. Confirm the deployed commit matches the intended commit; then hard-refresh or use a private window if stale browser assets persist.

No build step transforms these files. A successful upload does not prove that database migrations, Auth settings, or RLS are correct. Inspect browser console/network responses and Supabase logs when backend calls fail.

9. Configuration checklist for a new environment

1. Create a Supabase project and set `url` and public anon/publishable key in `supabase-config.js`.
2. Apply migrations 1–9 in sequence.
3. Configure Supabase Auth email confirmation, Site URL, and allowed redirect URLs for localhost and the live site.
4. Create an account through the storefront; promote the trusted initial admin using the SQL in `README.md`.
5. Sign in and check that the admin console passes role validation.
6. Replace demo catalog values; verify product prices, inventory, images, delivery areas/fees, contact values and site branding.
7. Place a low-value COD test order, verify order ownership and unique display reference, update its status in admin, and validate delivered-order rating eligibility before public launch.
8. Configure Cloudflare Pages root/output settings and deploy the production branch.

10. Troubleshooting

Symptom	Checks
`localhost` times out	Keep `py -m http.server 8000` running in the correct repo directory; open `http://localhost:8000`, not HTTPS.
Storefront loads but live data does not	Check `supabase-config.js`, browser console/network requests, Supabase project availability, and whether required migrations were applied.
Checkout says no delivery areas	Apply migration 2 and confirm active rows exist in `delivery_zones`.
Admin page is blank/denied	Sign in first; confirm the signed-in user's `profiles.role` is `admin`; ensure migrations and RLS policies are present. The obscure filename does not bypass the role check.
Product/settings image upload fails	Apply the corresponding storage migration; use JPG, PNG, or WebP within the configured 5 MB limit; confirm the account is admin.
Footer contact fields are absent/stale	Apply migration 8; save values in Admin → Settings; confirm the deployed browser loads the current `app.js`; inspect the `site_settings` row with id `main` and the read policy.
Password reset returns to the wrong page	Check Supabase Auth Site URL and redirect allow-list for the deployed domain.
Pages deployment has no build output	This is expected for the static site: no build command, root output directory `/`. Confirm the repository root is selected.

11. Change management

Keep SQL changes as ordered migration files. Update this guide and `README.md` when configuration or behavior changes. Avoid hand-editing production data through broad SQL updates; use the admin UI or narrow reviewed SQL. Commit source and migrations together so Cloudflare Pages and Supabase schema changes can be tracked independently.

Frostly - User Manual

This guide covers the customer storefront and administrator console. Screens and labels may appear in English or Bangla depending on the language selector. The examples assume the site has been connected to Supabase and its required migrations have been applied.

Contents

1. Open Frostly
2. Browse and choose products
3. Create an account and sign in
4. Manage your profile
5. Place a Cash on Delivery order
6. View orders and rate delivered products
7. Administrator access
8. Use the admin dashboard
9. Troubleshooting

1. Open Frostly

For local use, open a terminal in the project folder and run:

```
py -m http.server 8000
```

Keep that window open, then visit `http://localhost:8000`. To stop the local server, press `Ctrl+C`. The deployed store is opened at its Cloudflare Pages domain.

The storefront starts in English. Use the language dropdown to switch to Bangla; your choice is remembered in that browser.

2. Browse and choose products

1. Browse the home page, categories, best sellers, or all products.
2. Use search to find an item.
3. Select a product card to open its product details and larger image.
4. Use **Add to Cart** to add an item. Open the cart control to review items, quantities, subtotal, and checkout.
5. Use the heart control to manage your wishlist.

Product names, prices, stock, and images depend on the catalog configured by the store. Demo catalog data must be replaced by the operator before real sales.

3. Create an account and sign in

1. Open the account/profile control and choose **Create an account**.
2. Enter your name, email and password, then submit.
3. If email confirmation is enabled, open the confirmation email before signing in.
4. For returning customers, choose **Sign in** and enter your email and password.
5. If you cannot remember your password, choose the password-recovery link and follow the email instructions.

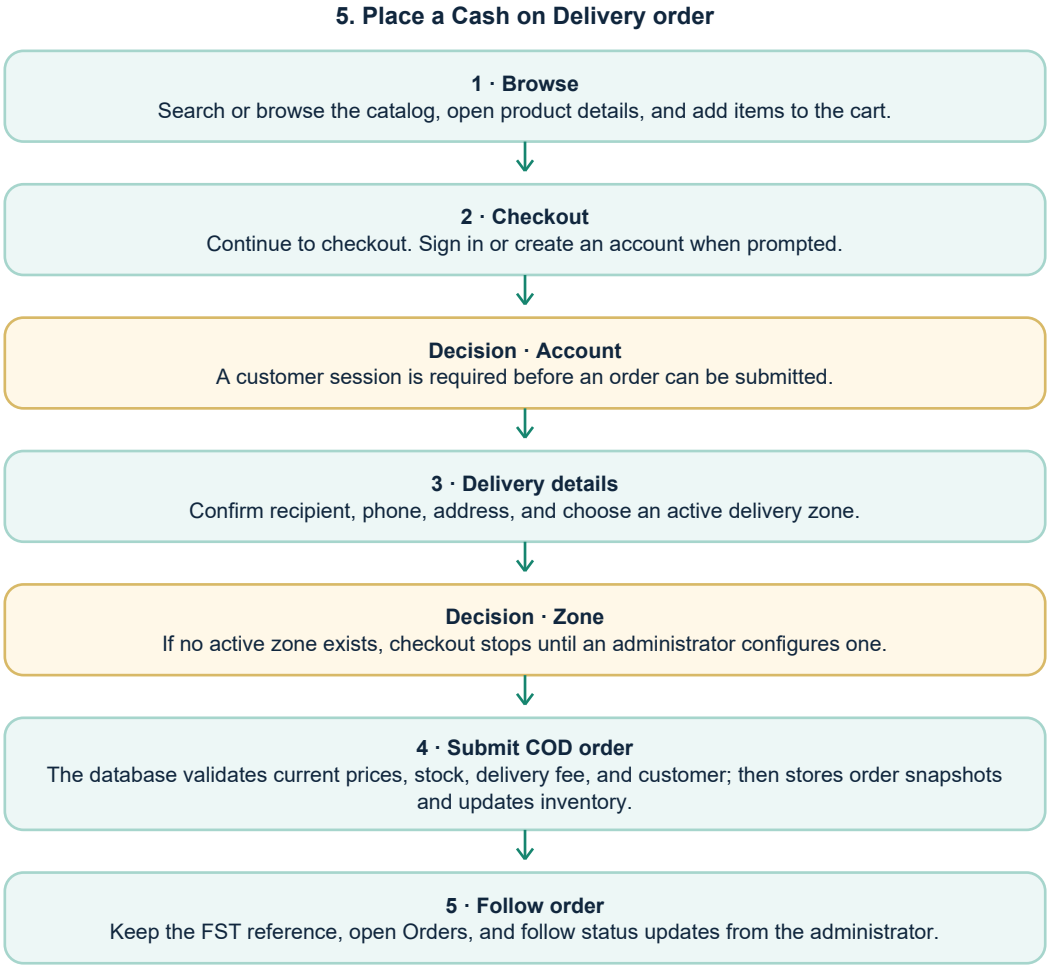
The exact confirmation steps depend on the Supabase Auth settings configured by the site operator.

4. Manage your profile

Open your account/profile panel after signing in. You can update your displayed name, phone number and profile picture, and change your password by entering your current password and a new password with confirmation. Use the saved-address/default-address controls available in the account/checkout experience to make delivery faster. Sign out from the account panel when finished on a shared device.

If an update reports an error, check that the account is signed in and that profile/storage migrations are installed. A profile photo must be a supported image type and within the configured upload limit.

5. Place a Cash on Delivery order



1. Open the cart and choose **Checkout**.
2. Sign in if prompted.
3. Confirm the recipient name, phone number, delivery address, delivery area and optional instructions.
4. Choose whether to save the address if that option is shown.
5. Review items, subtotal and delivery fee. The fee comes from the selected active delivery area.
6. Choose **Place order**. Cash on Delivery is the enabled payment method.
7. Keep the order reference shown after successful submission; use the **Orders** action to go to your orders.

The displayed total is an estimate until the database validates the current price, stock and delivery fee when the order is submitted. If checkout reports that no delivery areas are configured, the store administrator must activate delivery areas in the dashboard/Supabase first. Do not submit payment details; online card/mobile-wallet payment is not implemented.

6. View orders and rate delivered products

Use the separate Orders control to open the customer order drawer. Filter orders by their status, refresh the list, and select an order reference/card to expand its products and order details. Active/in-progress, delivered and cancelled orders may be shown in separate filters.

After an order is marked **Delivered**, an eligible customer can rate a product included in that order. Choose one to five stars and optionally write a short review. A customer can update their rating for that product. Reviews are public storefront content, so avoid including private information.

7. Administrator access

The administrator console is available at the console HTML filename distributed with the project. Its filename is not a security boundary. Access requires signing in with an account whose Supabase profile role is `admin`; direct URL access by a normal customer should be denied by the application and database policies.

The first administrator account must be created as a normal customer account and then promoted by a trusted operator in the Supabase SQL Editor. Follow the SQL in the repository `README.md`. Never share the SQL Editor with untrusted users and never expose a service-role key in the website.

8. Use the admin dashboard

Overview

The overview displays live order/sales and stock summaries. Select the low-stock product card to open the inventory view and update affected product stock/reorder information.

Orders

1. Open **Orders** and use search/status filters to locate an order.
2. Processing and delivered order shortcuts filter the list.
3. Review the customer, delivery address, items, totals and placed time.
4. Select an allowed status and save. Available statuses include awaiting confirmation, confirmed, preparing, packed, out for delivery, delivered and cancelled. The database validates status transitions and records history.
5. Refresh the dashboard if another administrator has changed data.

Products and inventory

1. Open **Products** and choose **New product** or edit an existing product.
2. Enter product names, category, price, stock and other required catalog information.
3. Upload a JPG, PNG, or WebP image up to 5 MB, or enter an image URL if the form allows it.
4. Save the product. Use the inventory view for stock adjustments; adjustments are recorded in inventory movement history.
5. Check that the product is active and the image is publicly accessible before expecting it to show in the storefront.

Delivery areas

Open the delivery area/zone section to add or edit an area name, delivery fee, and active state. A zone must be active for customers to select it at checkout. Confirm the actual fee before enabling the area. The initial migration seeds Inside Chattogram Town at Tk 80 and Outside Chattogram Town at Tk 120.

Store settings

Open **Settings** to edit the site name, logo, homepage hero slides, service area, phone, email and payment-method display text. Upload supported images where offered, edit slide text in both languages if shown, then use the relevant save button and wait for its saved confirmation. Refresh the storefront to confirm the public settings update.

Contact information appears in the storefront footer only when values are configured. Use real customer-facing contact details; placeholder data should be replaced. The payment-method footer text is informational and does not add a payment gateway.

Admin profile and sign-out

Open the admin name/avatar control in the dashboard header to view the signed-in administrator identity and choose **Sign out**. Admin role changes and password recovery are managed through the account/Supabase Auth process.

9. Troubleshooting

Problem	What to do
The local website will not open	Check that the Python server is still running in the Frostly project folder and use `http://localhost:8000`.
Sign-in or account creation fails	Check the email/password, confirmation email, internet connection and Supabase Auth configuration.
Checkout cannot continue	Sign in, fill all required fields, and confirm that the chosen area is active and has a delivery fee.
Stock or total changes at checkout	The final calculation uses current database prices and stock. Review the cart and try again if an item sold out.
Admin dashboard says access denied	Use the account specifically promoted to admin; a hidden/obscure page URL does not make an account an administrator.
Image upload fails	Use JPG, PNG or WebP under 5 MB and confirm the relevant storage migration is installed.
Changes do not appear on the live site	Confirm the settings save completed, the correct Supabase project is connected, and the latest Git commit deployed in Cloudflare Pages. Refresh without cache.
Review controls are missing	The order must be delivered, the product must be in that order, and migration `202610220009_product_ratings.sql` must be applied.

For technical setup, migration details, security notes and deployment configuration, see Project Documentation and the repository README.